DOCKER COMMAND CHEAT SHEET

Operations · Management · Monitoring · Networking · Storage

Quick-reference for DevOps & System Administrators

CONTAINER MANAGEMENT

Command	Purpose	Example
docker ps	List running containers	docker ps
docker ps -a	List ALL containers (running + stopped)	docker ps -a
docker ps -q	List only container IDs	docker ps -q
docker ps --filter status=exited	List only exited containers	docker ps --filter status=exited
docker run IMAGE	Create & start a container from image	docker run nginx
docker run -d IMAGE	Run container in background (detached)	docker run -d nginx
docker run -it IMAGE bash	Run interactive terminal in container	docker run -it ubuntu bash
docker run --name NAME IMAGE	Assign a custom name to container	docker run --name web nginx
docker run -p HOST:CTR IMAGE	Map host port to container port	docker run -p 8080:80 nginx
docker run -v HOST:CTR IMAGE	Mount host volume into container	docker run -v /data:/app nginx
docker run --rm IMAGE	Auto-remove container when it exits	docker run --rm alpine echo hi
docker run -e KEY=VAL IMAGE	Set environment variable	docker run -e APP_ENV=prod nginx
docker run --restart always IMAGE	Auto-restart policy (always/unless-stopped/on-failure)	docker run --restart always IMAGE
docker start CONTAINER	Start a stopped container	docker start web
docker stop CONTAINER	Gracefully stop a running container (SIGTERM)	docker stop web
docker restart CONTAINER	Stop then start a container	docker restart web

docker kill CONTAINER	Forcefully kill container immediately (SIGKILL)	docker kill web
docker pause CONTAINER	Suspend all processes in container	docker pause web
docker unpause CONTAINER	Resume suspended container	docker unpause web
docker rm CONTAINER	Remove a stopped container	docker rm web
docker rm -f CONTAINER	Force remove a running container	docker rm -f web
docker rm $(docker ps -aq)	Remove ALL stopped containers	docker rm $(docker ps -aq)
docker rename OLD NEW	Rename an existing container	docker rename web app
docker update --cpus 2 CONTAINER	Update CPU/memory limits live	docker update --cpus 2 web
docker commit CONTAINER IMAGE	Save container state as new image	docker commit web myimage:v2
docker cp SRC CONTAINER:DEST	Copy files from host to container	docker cp file.txt web:/app/
docker cp CONTAINER:SRC DEST	Copy files from container to host	docker cp web:/app/log.txt .
docker diff CONTAINER	Show changes to container filesystem	docker diff web
docker wait CONTAINER	Block until container stops; print exit code	docker wait web
docker export CONTAINER > file.tar	Export container filesystem as tar	docker export CONTAINER > file.tar
docker import file.tar IMAGE	Import tar as new image	docker import file.tar IMAGE

EXEC & INTERACTIVE ACCESS

Command	Purpose	Example
docker exec CONTAINER CMD	Run command in running container	docker exec web ls /app
docker exec -it CONTAINER bash	Open interactive bash shell in container	docker exec -it web bash
docker exec -it CONTAINER sh	Open sh shell (for Alpine/slim images)	docker exec -it web sh

docker exec -u root CONTAINER CMD	Execute as specific user	docker exec -u root web id
docker exec -e KEY=VAL CTR CMD	Execute with extra env var	docker exec -e KEY=VAL CTR CMD
docker attach CONTAINER	Attach to running container's stdin/stdout	docker attach web
docker run -it IMAGE CMD	Run interactive command in new container	docker run -it alpine sh

MONITORING & LOGGING

Command	Purpose	Example
docker logs CONTAINER	Show stdout/stderr logs	docker logs web
docker logs -f CONTAINER	Follow (tail) logs in real time	docker logs -f web
docker logs --tail 100 CONTAINER	Show last N lines of logs	docker logs --tail 100 web
docker logs --since 1h CONTAINER	Show logs from the past 1 hour	docker logs --since 1h web
docker logs --until 2024-01-01 CTR	Show logs until a timestamp	docker logs --until 2024-01-01 CTR
docker logs -t CONTAINER	Show logs with timestamps	docker logs -t web
docker stats	Live CPU/RAM/network/disk usage for ALL containers	docker stats
docker stats CONTAINER	Live stats for a specific container	docker stats web
docker stats --no-stream	One-shot snapshot of all stats (no live update)	docker stats --no-stream
docker stats --format TABLE	Custom format stats output	docker stats --format "{{.Name}}\t{{.CPUPerc}}"
docker top CONTAINER	Show processes running inside container	docker top web
docker events	Stream real-time Docker events	docker events
docker events --filter type=container	Filter events by type	docker events --filter type=container
docker inspect CONTAINER	Full JSON metadata of container	docker inspect web
docker inspect -f FIELD CONTAINER	Extract specific field from inspect	docker inspect -f '{{.State.Status}}' web

docker port CONTAINER	Show port mappings for container	docker port web

IMAGE MANAGEMENT

Command	Purpose	Example
docker images	List all local images	docker images
docker images -a	List all including intermediate layers	docker images -a
docker pull IMAGE	Download image from registry	docker pull ubuntu:22.04
docker pull IMAGE:TAG	Pull specific tag/version	docker pull nginx:1.25
docker push IMAGE	Push image to registry	docker push myuser/app:v1
docker build -t NAME:TAG .	Build image from Dockerfile in current dir	docker build -t myapp:v1 .
docker build -f FILE -t NAME .	Build using custom Dockerfile path	docker build -f Dockerfile.prod -t app .
docker build --no-cache -t NAME .	Build without using cache	docker build --no-cache -t NAME .
docker build --build-arg KEY=VAL .	Pass build argument	docker build --build-arg ENV=prod .
docker tag IMAGE NEW_TAG	Tag an image with new name/version	docker tag myapp:v1 myapp:latest
docker rmi IMAGE	Remove a local image	docker rmi nginx
docker rmi -f IMAGE	Force remove image (even if in use)	docker rmi -f nginx
docker image prune	Remove dangling (unused) images	docker image prune
docker image prune -a	Remove ALL unused images	docker image prune -a
docker save IMAGE > file.tar	Save image as tar archive	docker save myapp > myapp.tar
docker load < file.tar	Load image from tar archive	docker load < myapp.tar
docker history IMAGE	Show image layer history	docker history nginx
docker search TERM	Search Docker Hub for images	docker search nginx

docker image inspect IMAGE	Full JSON metadata of image	docker image inspect nginx
docker image ls --filter dangling=true	List dangling images only	docker image ls --filter dangling=true

NETWORK MANAGEMENT

Command	Purpose	Example
docker network ls	List all networks	docker network ls
docker network create NAME	Create a new bridge network	docker network create mynet
docker network create --driver NAME	Create network with specific driver	docker network create --driver overlay mynet
docker network inspect NAME	Inspect network details & connected containers	docker network inspect mynet
docker network connect NET CTR	Connect container to network	docker network connect mynet web
docker network disconnect NET CTR	Disconnect container from network	docker network disconnect mynet web
docker network rm NAME	Remove a network	docker network rm mynet
docker network prune	Remove ALL unused networks	docker network prune
docker run --network NAME IMAGE	Run container on specific network	docker run --network mynet nginx
docker run --network host IMAGE	Use host network (no isolation)	docker run --network host nginx
docker run --network none IMAGE	No network access (fully isolated)	docker run --network none nginx
docker run -p 80:80 IMAGE	Publish container port 80 to host port 80	docker run -p 80:80 nginx
docker run -p 127.0.0.1:80:80 IMAGE	Bind to localhost only	docker run -p 127.0.0.1:80:80 IMAGE
docker run -P IMAGE	Publish all exposed ports to random host ports	docker run -P nginx

docker run --link CTR:ALIAS IMAGE	Link containers (legacy — prefer networks)	docker run --link db:database web
docker run --add-host HOST:IP IMAGE	Add custom host entry to /etc/hosts	docker run --add-host HOST:IP IMAGE
docker run --dns 8.8.8.8 IMAGE	Use custom DNS server	docker run --dns 8.8.8.8 IMAGE

VOLUME & STORAGE MANAGEMENT

Command	Purpose	Example
docker volume ls	List all volumes	docker volume ls
docker volume create NAME	Create a named volume	docker volume create dbdata
docker volume inspect NAME	Inspect volume details	docker volume inspect dbdata
docker volume rm NAME	Remove a named volume	docker volume rm dbdata
docker volume prune	Remove ALL unused volumes	docker volume prune
docker run -v VOL:/path IMAGE	Mount named volume into container	docker run -v dbdata:/var/lib/mysql mysql
docker run -v /host:/ctr IMAGE	Bind mount host directory	docker run -v /data:/app nginx
docker run --mount type=bind,...	Explicit mount syntax (recommended)	docker run --mount type=bind,...
docker run --mount type=volume,...	Named volume with explicit syntax	docker run --mount type=volume,...
docker run --mount type=tmpfs,...	Temporary in-memory filesystem	docker run --mount type=tmpfs,...
docker run -v VOL:/path:ro IMAGE	Mount volume as read-only	docker run -v conf:/etc/nginx:ro nginx
docker run --volumes-from CTR IMAGE	Share volumes from another container	docker run --volumes-from db backup

DOCKER COMPOSE

Command	Purpose	Example
docker compose up	Create & start all services	docker compose up
docker compose up -d	Start services in detached mode	docker compose up -d
docker compose up --build	Rebuild images before starting	docker compose up --build
docker compose up SERVICE	Start a specific service only	docker compose up web
docker compose down	Stop & remove containers, networks	docker compose down
docker compose down -v	Also remove named volumes	docker compose down -v
docker compose down --rmi all	Also remove all built images	docker compose down --rmi all
docker compose stop	Stop services without removing them	docker compose stop
docker compose start	Start stopped compose services	docker compose start
docker compose restart	Restart all services	docker compose restart
docker compose restart SERVICE	Restart a specific service	docker compose restart web
docker compose ps	List compose services & their status	docker compose ps
docker compose logs	Show logs for all services	docker compose logs
docker compose logs -f SERVICE	Follow logs for a specific service	docker compose logs -f web
docker compose exec SERVICE CMD	Run command inside a service container	docker compose exec web bash
docker compose run SERVICE CMD	Run one-off command in a new container	docker compose run web python manage.py migrate
docker compose build	Build or rebuild service images	docker compose build
docker compose build SERVICE	Build a specific service image	docker compose build web
docker compose pull	Pull latest images for all services	docker compose pull
docker compose config	Validate & display compose config	docker compose config

docker compose scale S=N	Scale service to N instances (v1 only)	docker compose scale web=3
docker compose up --scale S=N	Scale service (v2 syntax)	docker compose up --scale web=3
docker compose top	Show processes in each service	docker compose top
docker compose pause	Pause all services	docker compose pause
docker compose unpause	Unpause all services	docker compose unpause
docker compose images	List images used by compose	docker compose images
docker compose version	Show Docker Compose version	docker compose version
docker compose -f FILE up	Use alternative compose file	docker compose -f docker-compose.prod.yml up
docker compose --env-file FILE up	Use custom env file	docker compose --env-file .env.prod up

SYSTEM & CLEANUP

Command	Purpose	Example
docker info	Display system-wide Docker info	docker info
docker version	Show Docker client & server version	docker version
docker system df	Show disk usage by images, containers, volumes	docker system df
docker system df -v	Detailed disk usage breakdown	docker system df -v
docker system prune	Remove all unused data (stopped ctrs, nets, imgs)	docker system prune
docker system prune -a	Also remove unused images (not just dangling)	docker system prune -a
docker system prune -a --volumes	Full nuke: images + volumes + containers	docker system prune -a --volumes
docker container prune	Remove all stopped containers	docker container prune
docker image prune	Remove dangling images	docker image prune

docker image prune -a	Remove all unused images	docker image prune -a
docker network prune	Remove unused networks	docker network prune
docker volume prune	Remove unused volumes	docker volume prune
docker builder prune	Remove build cache	docker builder prune
docker builder prune -a	Remove all build cache	docker builder prune -a

REGISTRY & AUTHENTICATION

Command	Purpose	Example
docker login	Log in to Docker Hub	docker login
docker login REGISTRY	Log in to private registry	docker login registry.example.com
docker logout	Log out from Docker Hub	docker logout
docker logout REGISTRY	Log out from private registry	docker logout registry.example.com
docker pull USER/IMAGE:TAG	Pull from Docker Hub	docker pull nginx:latest
docker push USER/IMAGE:TAG	Push to Docker Hub	docker push myuser/myapp:v1
docker tag IMAGE REGISTRY/IMAGE:TAG	Tag for private registry	docker tag app registry.io/myapp:v1
docker push REGISTRY/IMAGE:TAG	Push to private registry	docker push registry.io/myapp:v1
docker search IMAGE	Search Docker Hub	docker search nginx
docker search --filter stars=100 IMG	Search with star-count filter	docker search --filter stars=100 nginx

HEALTH CHECKS & RESOURCE LIMITS

Command	Purpose	Example

docker run --health-cmd CMD IMAGE	Add health check command	docker run --health-cmd "curl -f http://localhost/" nginx
docker run --health-interval=30s	Set health check interval	docker run --health-interval=30s
docker run --health-timeout=10s	Set health check timeout	docker run --health-timeout=10s
docker run --health-retries=3	Set health check retry count	docker run --health-retries=3
docker run --no-healthcheck IMAGE	Disable health check	docker run --no-healthcheck IMAGE
docker inspect --format '{{.State.Health.Status}}' CTR	Check health status	docker inspect --format '{{.State.Health.Status}}' CTR
docker run --cpus 2 IMAGE	Limit to 2 CPU cores	docker run --cpus 2 nginx
docker run --cpu-shares 512 IMAGE	Relative CPU weight (default 1024)	docker run --cpu-shares 512 nginx
docker run -m 512m IMAGE	Limit memory to 512 MB	docker run -m 512m nginx
docker run --memory-swap 1g IMAGE	Limit memory + swap total	docker run --memory-swap 1g nginx
docker run --pids-limit 100 IMAGE	Limit number of processes	docker run --pids-limit 100 IMAGE
docker run --ulimit nofile=1024 IMAGE	Set ulimit (open files etc.)	docker run --ulimit nofile=1024 IMAGE
docker update --cpus 2 CONTAINER	Update CPU limit on running container	docker update --cpus 2 web
docker update -m 1g CONTAINER	Update memory limit on running container	docker update -m 1g web



DOCKER SWARM (ORCHESTRATION)

Command	Purpose	Example
docker swarm init	Initialize a Swarm manager on this node	docker swarm init
docker swarm join --token TOKEN HOST	Join swarm as a worker node	docker swarm join --token TOKEN HOST

docker swarm join-token worker	Show worker join token	docker swarm join-token worker
docker swarm join-token manager	Show manager join token	docker swarm join-token manager
docker swarm leave	Leave the swarm	docker swarm leave
docker swarm leave --force	Force leave (if manager node)	docker swarm leave --force
docker node ls	List swarm nodes	docker node ls
docker node inspect NODEID	Inspect a swarm node	docker node inspect NODEID
docker node promote NODE	Promote worker to manager	docker node promote node2
docker node demote NODE	Demote manager to worker	docker node demote node2
docker node rm NODE	Remove a node from swarm	docker node rm node2
docker service create IMAGE	Create a swarm service	docker service create --name web nginx
docker service ls	List swarm services	docker service ls
docker service ps SERVICE	List tasks (replicas) of a service	docker service ps web
docker service inspect SERVICE	Inspect service details	docker service inspect web
docker service update SERVICE	Update a running service	docker service update --replicas 3 web
docker service scale S=N	Scale service to N replicas	docker service scale web=3
docker service logs SERVICE	Show logs for a swarm service	docker service logs web
docker service rm SERVICE	Remove a swarm service	docker service rm web
docker stack deploy -c FILE NAME	Deploy stack from compose file	docker stack deploy -c docker-compose.yml myapp
docker stack ls	List deployed stacks	docker stack ls
docker stack ps STACK	List tasks in a stack	docker stack ps myapp

docker stack services STACK	List services in a stack	docker stack services myapp
docker stack rm STACK	Remove a deployed stack	docker stack rm myapp

DOCKERFILE INSTRUCTIONS REFERENCE

Flag / Option	Description
FROM image:tag	Base image to build from (must be first instruction)
RUN command	Execute command during build (creates a new layer)
CMD ["cmd","arg"]	Default command when container starts (overridable)
ENTRYPOINT ["cmd"]	Fixed entrypoint command (CMD becomes its arguments)
COPY src dest	Copy files from build context into image
ADD src dest	Like COPY but also handles URLs and auto-extracts tars
WORKDIR /path	Set working directory for subsequent instructions
ENV KEY=VALUE	Set environment variable in image
ARG NAME=default	Define build-time variable (docker build --build-arg)
EXPOSE port	Document which port the container listens on
VOLUME ["/data"]	Create mount point / declare anonymous volume
USER username	Switch to this user for subsequent instructions
LABEL key=value	Add metadata to image
HEALTHCHECK CMD ...	Define container health check command
ONBUILD instruction	Trigger instruction when image is used as a base
STOPSIGNAL signal	Set signal to stop the container (default SIGTERM)
SHELL ["cmd","arg"]	Override default shell for RUN instructions
MAINTAINER name	Author info (deprecated — use LABEL instead)

docker run — COMMON FLAGS REFERENCE

Flag / Option	Description

-d	Detached mode: run in background
-it	Interactive terminal (combine -i and -t)
-i	Keep STDIN open
-t	Allocate a pseudo-TTY
--name NAME	Assign a name to the container
-p HOST:CTR	Publish container port to host
-P	Publish all exposed ports to random host ports
-e KEY=VALUE	Set environment variable
--env-file FILE	Read environment variables from file
-v HOST:CTR[:ro]	Bind mount or named volume (:ro for read-only)
--mount type=...	Explicit, verbose mount syntax (recommended)
--network NAME	Connect to named network
--restart always	Restart policy: always / unless-stopped / on-failure
--rm	Automatically remove container after it exits
--cpus N	Limit to N CPU cores
-m SIZE	Memory limit e.g. 512m, 1g
--memory-swap SIZE	Total memory+swap limit
-u USER	Run as specified user or UID
--workdir /path	Working directory inside container
--entrypoint CMD	Override the image's ENTRYPOINT
--privileged	Give container full host privileges (use with caution)
--read-only	Mount container root filesystem as read-only
--hostname NAME	Set container hostname
--add-host HOST:IP	Add entry to /etc/hosts
--dns IP	Set custom DNS server
--label key=val	Attach metadata label to container
--log-driver NAME	Logging driver: json-file, syslog, fluentd, none...
--log-opt KEY=VAL	Logging driver options

--security-opt ...	Security options (seccomp, apparmor, no-new-privileges)
--cap-add CAP	Add Linux capability
--cap-drop CAP	Drop Linux capability
--device /dev/...	Allow access to host device
--pid host	Share host PID namespace
--ipc host	Share host IPC namespace
--ulimit nofile=1024	Set ulimit for the container
--sysctl net.ipv4...=1	Set kernel parameter
-w /path	Working directory shorthand
--init	Run init process as PID 1 (reaps zombie processes)

HANDY ONE-LINERS & POWER COMMANDS

What it does	Command
Stop ALL running containers	**docker stop $(docker ps -q)**
Remove ALL stopped containers	**docker rm $(docker ps -aq)**
Remove ALL images	**docker rmi $(docker images -q)**
Full system cleanup (nuclear option)	**docker system prune -a --volumes -f**
Enter running container as root	**docker exec -u root -it CONTAINER bash**
Copy file into container	**docker cp ./file.txt CONTAINER:/app/**
Show container IP address	**docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' CONTAINER**
Show all port mappings	**docker ps --format 'table {{.Names}}\t{{.Ports}}'**
Tail logs from multiple containers	**docker compose logs -f web db redis**
Remove exited containers	**docker rm $(docker ps -q -f status=exited)**
Pull & run latest image in one command	**docker run --rm --pull=always nginx**
Watch container stats continuously	**watch -n 1 docker stats --no-stream**
Show disk usage by container	**docker system df -v**
List container names and their images	**docker ps --format 'table {{.Names}}\t{{.Image}}'**

Get container start time	**docker inspect -f '{{.State.StartedAt}}' CONTAINER**
Show env vars in container	**docker exec CONTAINER env**
View container resource limits	**docker inspect --format '{{.HostConfig.Memory}}' CONTAINER**
Export container logs to file	**docker logs CONTAINER > app.log 2>&1**
Run temporary busybox for debugging	**docker run --rm -it busybox sh**
Run Alpine Linux for quick testing	**docker run --rm -it alpine sh**
Build & tag in one step	**docker build -t myapp:$(git rev-parse --short HEAD) .**

FORMATTING OUTPUT WITH --format

Go template	Shows
docker ps --format '{{.Names}}'	Container names only
docker ps --format '{{.ID}}'	Container IDs only
docker ps --format '{{.Image}}'	Image used
docker ps --format '{{.Status}}'	Status
docker ps --format '{{.Ports}}'	Port mappings
docker ps --format 'table {{.Names}}\t{{.Status}}'	Formatted table
docker images --format '{{.Repository}}:{{.Tag}}'	image:tag pairs
docker images --format '{{.Size}}'	Image sizes
docker stats --format '{{.Name}} {{.CPUPerc}}'	Name + CPU %
docker inspect -f '{{.State.Status}}'	Container status
docker inspect -f '{{.NetworkSettings.IPAddress}}'	Container IP
docker inspect -f '{{.HostConfig.RestartPolicy.Name}}'	Restart policy